Semester: III
Academic Year: 2026-27
Name of Instructor:
Ms. Rujata Chaudhari
Ms. Rucha Kulkarni
Mr. Mandar Ganjapurkar

Name of Student:
Student ID:
Class / Branch: SE IT A/B/C
Subject: Mini Project FSJP Lab

Assignment No: 2

CO ID	CO Statements	Bloom's Taxonomy
2343611.1	Apply the setup and configuration procedures to establish a full-stack Java development environment.	L3
2343611.2	Develop and implement responsive user interfaces using HTML, CSS, and React.js.	L3
2343611.3	Implement RESTful APIs using the Spring Boot framework for application development.	L3
2343611.4	Apply CRUD operations to integrate and manage data in a MySQL database.	L3
2343611.5	Integrate and use front-end and back-end components through secure API endpoints	L3
2343611.6	Deploy and demonstrate a full-stack application on a cloud platform as a functional solution.	L3

Q.1 Develop a Student Record Management System using React.js for the frontend, Spring Boot for backend development, and MySQL for database management. Implement REST APIs to add, view, update, and delete student records, configure database connectivity using Spring Data JPA, integrate the frontend with the backend APIs, and deploy the application on a suitable hosting platform. Verify the correct functioning of the application and the storage and retrieval of student records. (CO3,CO4,CO5,CO6,BL3)

Solution:

Student Record Management System
Technology Stack: React.js, Spring Boot, MySQL, Spring Data JPA, REST API

Objective: To develop and deploy a full-stack web application that manages student records by integrating a React.js frontend, a Spring Boot backend, and a MySQL database.
1. System architecture
Frontend — React.js

Student form, records table, user interactions

Backend — Spring Boot

REST Controller → Repository → Entity

Database — MySQL

Stores student records permanently

2. Create the MySQL database
Execute the following SQL commands in MySQL Workbench.

```
CREATE DATABASE studentdb;

USE studentdb;

CREATE TABLE students (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    department VARCHAR(100) NOT NULL
);
```

The students table stores the student ID, name, email, and department.

3. Develop the Spring Boot backend

Create a Spring Boot project using Spring Initializr with these dependencies:

- Spring Web
- Spring Data JPA
- MySQL Driver

A. Configure application.properties

File: src/main/resources/application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=YOUR_MYSQL_PASSWORD

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
server.port=8080
```

Replace YOUR_MYSQL_PASSWORD with the actual MySQL password. Do not share real database credentials or commit them to a public repository.

B. Create the Student entity

File: Student.java

```
package com.example.studentmanagement;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String department;

    public Student() {}

    public Long getId() {
        return id;
    }

    public String getName() {
```

```
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }
}
```

The entity maps the Java Student class to the MySQL students table.

C. Create the repository

File: StudentRepository.java

```
package com.example.studentmanagement;

import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

JpaRepository provides built-in methods to save, retrieve, update, and delete records.

D. Create the REST controller

File: StudentController.java

```
package com.example.studentmanagement;

import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
```

```
@RequestMapping("/api/students")
@CrossOrigin(origins = {
   "http://localhost:3000",
   "http://localhost:5173"
})
public class StudentController {

   private final StudentRepository repository;

   public StudentController(StudentRepository repository) {
      this.repository = repository;
   }

   @PostMapping
   public Student addStudent(@RequestBody Student student) {
      return repository.save(student);
   }

   @GetMapping
   public List<Student> getStudents() {
      return repository.findAll();
   }

   @PutMapping("/{id}")
   public Student updateStudent(
         @PathVariable Long id,
         @RequestBody Student updatedStudent) {

      Student student = repository.findById(id)
         .orElseThrow(() ->
            new RuntimeException("Student not found"));

      student.setName(updatedStudent.getName());
      student.setEmail(updatedStudent.getEmail());
      student.setDepartment(updatedStudent.getDepartment());

      return repository.save(student);
   }

   @DeleteMapping("/{id}")
   public String deleteStudent(@PathVariable Long id) {
      if (!repository.existsById(id)) {
         throw new RuntimeException("Student not found");
```

```
        }

        repository.deleteById(id);
        return "Student deleted successfully";
    }
}
```

Ensure that the main Spring Boot application class is in the com.example.studentmanagement package or a parent package.

4. Develop the React.js frontend

Create a React project and install the required dependencies.

```
npx create-react-app student-frontend
cd student-frontend
npm start
```

Create or replace src/App.js with the following code.

```
import React, { useEffect, useState } from "react";

const API = "http://localhost:8080/api/students";

function App() {
  const [students, setStudents] = useState([]);
  const [form, setForm] = useState({
    name: "",
    email: "",
    department: ""
  });
  const [editId, setEditId] = useState(null);
  const [error, setError] = useState("");

  const loadStudents = async () => {
    try {
      const response = await fetch(API);
      if (!response.ok) throw new Error("Unable to load students");
      setStudents(await response.json());
      setError("");
    } catch {
      setError("Cannot connect to backend. Check Spring Boot.");
    }
  };

  useEffect(() => {
    loadStudents();
  }, []);
```

```
const handleChange = (e) => {
  setForm({ ...form, [e.target.name]: e.target.value });
};

const handleSubmit = async (e) => {
  e.preventDefault();

  try {
    const response = await fetch(
      editId ? `${API}/${editId}` : API,
      {
        method: editId ? "PUT" : "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(form)
      }
    );

    if (!response.ok) throw new Error("Unable to save student");

    setForm({ name: "", email: "", department: "" });
    setEditId(null);
    await loadStudents();
  } catch (err) {
    setError(err.message);
  }
};

const editStudent = (student) => {
  setEditId(student.id);
  setForm({
    name: student.name,
    email: student.email,
    department: student.department
  });
};

const deleteStudent = async (id) => {
  if (!window.confirm("Delete this student?")) return;

  try {
    const response = await fetch(`${API}/${id}`, {
      method: "DELETE"
```

```
    });

    if (!response.ok) throw new Error("Unable to delete student");

    await loadStudents();
  } catch (err) {
    setError(err.message);
  }
};

return (
  <div style={{ maxWidth: 800, margin: "30px auto",
              fontFamily: "Arial" }}>
    <h2>Student Record Management System</h2>

    {error && <p style={{ color: "red" }}>{error}</p>}

    <form onSubmit={handleSubmit}>
      <input
        name="name"
        placeholder="Student Name"
        value={form.name}
        onChange={handleChange}
        required
      />

      <input
        name="email"
        type="email"
        placeholder="Email Address"
        value={form.email}
        onChange={handleChange}
        required
      />

      <input
        name="department"
        placeholder="Department"
        value={form.department}
        onChange={handleChange}
        required
      />
```

```
  <button type="submit">
    {editId ? "Update Student" : "Add Student"}
  </button>

  {editId && (
    <button type="button" onClick={() => {
      setEditId(null);
      setForm({ name: "", email: "", department: "" });
    }}>
      Cancel
    </button>
  )}
</form>

<h3>Student Records</h3>

<table border="1" cellPadding="8" cellSpacing="0"
       width="100%">
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Email</th>
      <th>Department</th>
      <th>Actions</th>
    </tr>
  </thead>

  <tbody>
    {students.map((student) => (
      <tr key={student.id}>
        <td>{student.id}</td>
        <td>{student.name}</td>
        <td>{student.email}</td>
        <td>{student.department}</td>
        <td>
          <button onClick={() => editStudent(student)}>
            Edit
          </button>
          <button onClick={() => deleteStudent(student.id)}>
            Delete
          </button>
        </td>
```

```
          </tr>
        ))}
      </tbody>
    </table>
  </div>
 );
}

export default App;
```

How the frontend works:

- The form collects the student's name, email, and department.
- POST adds a new student to the database.
- GET retrieves all students and displays them in a table.
- PUT updates an existing student's information.
- DELETE removes a student record.

The student ID is generated automatically by MySQL and displayed in the table.

5. Integrate and test the application

Start MySQL, then run the Spring Boot backend:

mvn spring-boot:run

In a second terminal, start React:

npm start

Open http://localhost:3000 in your browser.

Test the REST APIs using Postman as follows.

Operation	HTTP method	Endpoint
Add student	POST	http://localhost:8080/api/students
View students	GET	http://localhost:8080/api/students
Update student	PUT	http://localhost:8080/api/students/1
Delete student	DELETE	http://localhost:8080/api/students/1

For the POST request, select Body → raw → JSON in Postman and enter:

```
{
 "name": "Rahul Sharma",
 "email": "rahul@example.com",
 "department": "Information Technology"
}
```

For the PUT request, use the same JSON format with the updated details.

Expected result: the API returns the saved student record, and the React table displays the record retrieved from MySQL.

6. Deploy the application

The application can be deployed using a cloud hosting platform and a managed MySQL database.

Step 1: Package the Spring Boot backend

Run the following command inside the backend project directory:

mvn clean package

This generates a JAR file inside the target directory. Run it locally to verify the build:

java -jar target/studentmanagement-0.0.1-SNAPSHOT.jar

The actual JAR filename may differ depending on the project configuration.

Step 2: Deploy the backend and database

1. Create a MySQL database on a cloud database service.
2. Deploy the Spring Boot application to a suitable Java hosting platform.
3. Configure the database URL, username, and password using environment variables.
4. Update the backend CORS configuration to allow requests from the deployed frontend domain.

Step 3: Deploy the React frontend

Build the frontend:

npm run build

Deploy the generated static files to a suitable frontend hosting service.

Replace the local API address in App.js with the deployed backend URL, for example:

const API = "https://your-backend-domain.com/api/students";

Use the actual URL assigned by the hosting platform. Rebuild and redeploy the frontend after changing the API address.

Important: The frontend, backend, and database must be configured to communicate with each other. Do not use localhost for the cloud database or backend URL in the deployed frontend.

7. Final verification checklist

Test	Expected result
Add a student	Student is stored in MySQL
View student records	Records appear in the React table
Update a student	Updated information is displayed
Delete a student	Student record is removed
Restart backend	Existing records remain in MySQL
Access deployed application	Frontend communicates with the deployed backend

For production deployment, use secure environment variables, HTTPS, appropriate access controls, and managed database credentials. Avoid using spring.jpa.hibernate.ddl-auto=update as a substitute for controlled database migrations in production.